Microsoft

RAG with JavaScript

From Concepts to Code

aka.ms/ragjs/slides

Pamela Fox

Principal Cloud Advocate
www.pamelafox.org

The limitations of LLMs

You

Write a model class using latest version of Flask SQL Alchemy

Assistant

Sure! Here's an example of a model class using the latest version of Flask SQLAlchemy:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True)
    email = db.Column(db.String(120), unique=True)
```

Outdated public knowledge

You

Do my company perks cover underwater activities?

Assistant

To determine whether your company perks cover underwater activities, you should consult your employee benefits package or reach out to your human resources department. They will be able to provide you with detailed information regarding the specific perks and benefits offered by your company, including any coverage for underwater activities.

No internal knowledge

Integrating domain knowledge

Fine tuning

Learn new skills (permanently)

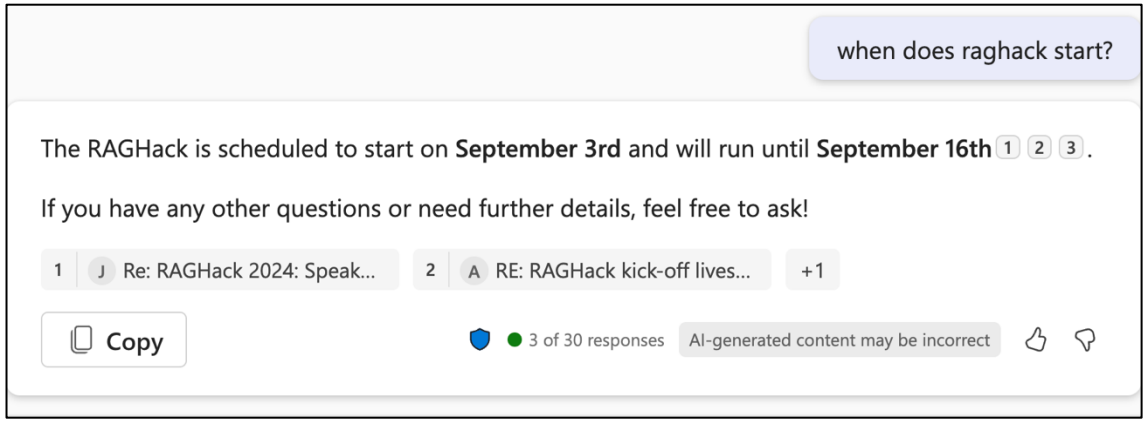
High cost, time

Retrieval Augmented Generation

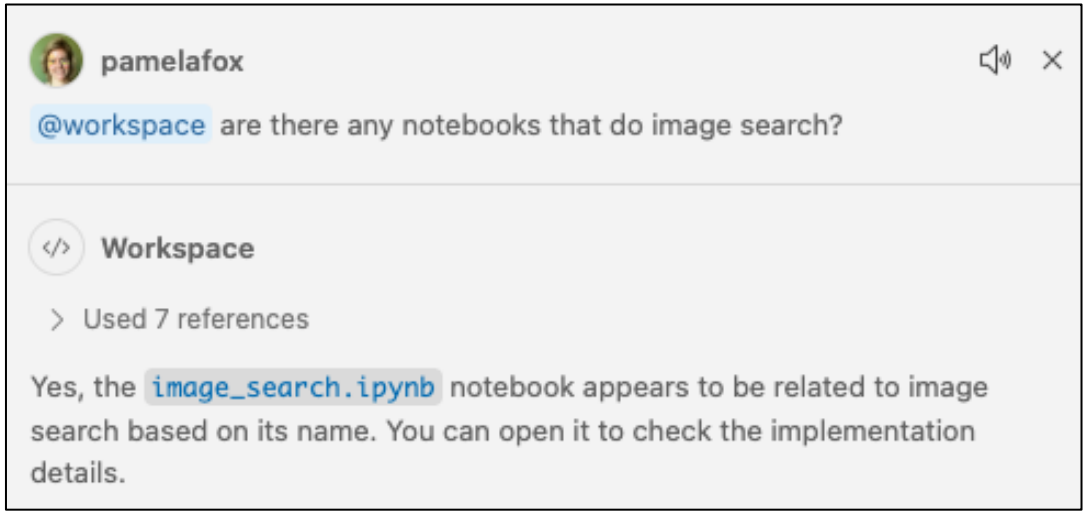
Learn new facts (temporarily)

RAG in the wild

Teams Copilot (RAG on your chats)

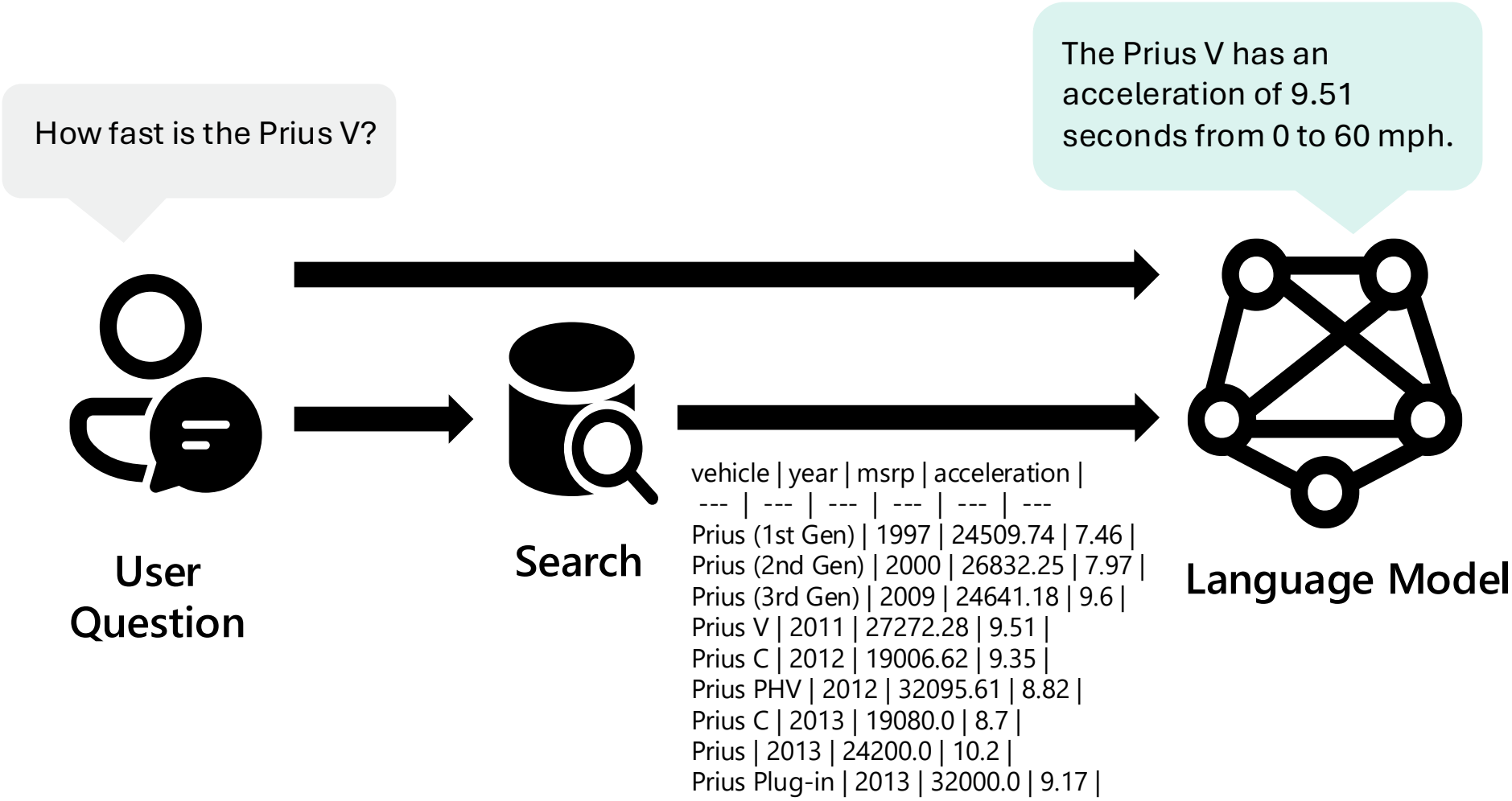


GitHub Copilot (RAG on VS Code workspace)

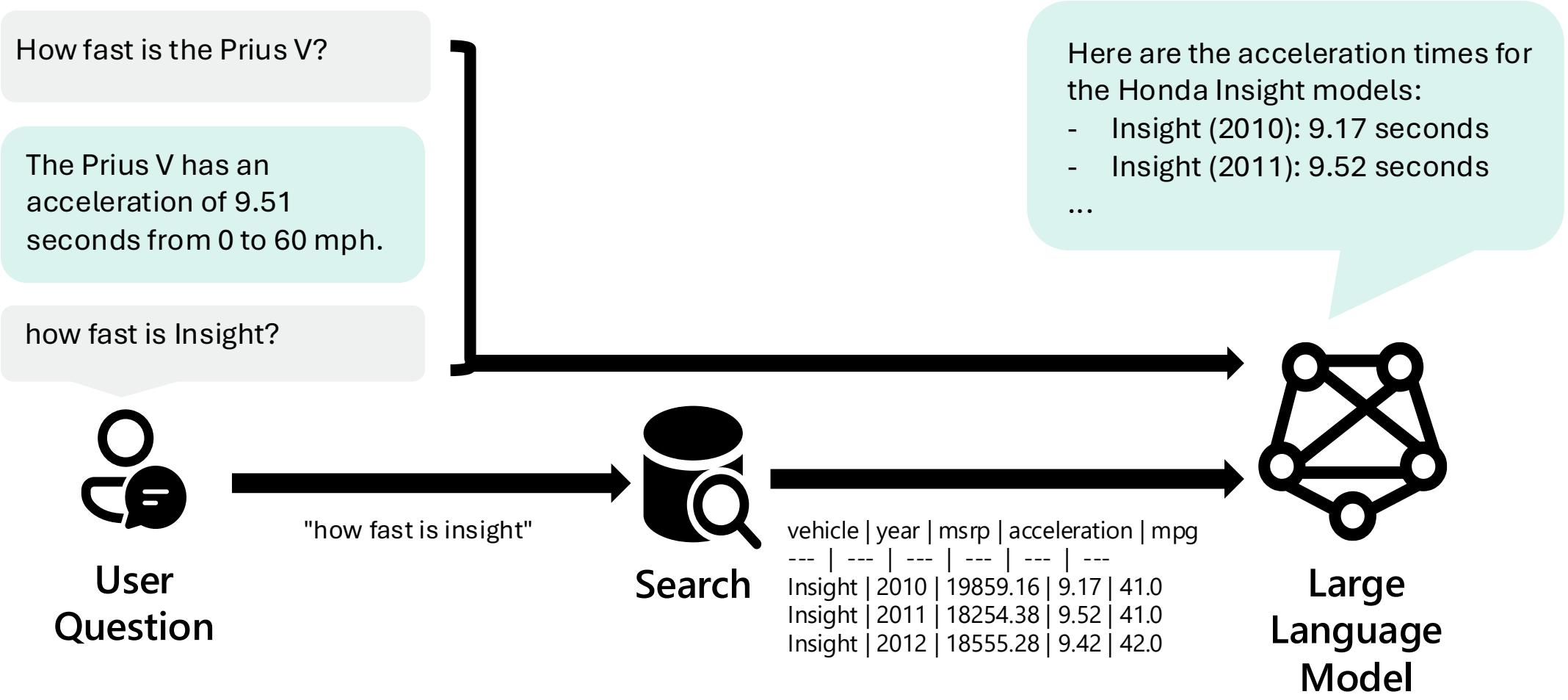


Copilot
(RAG on the web)

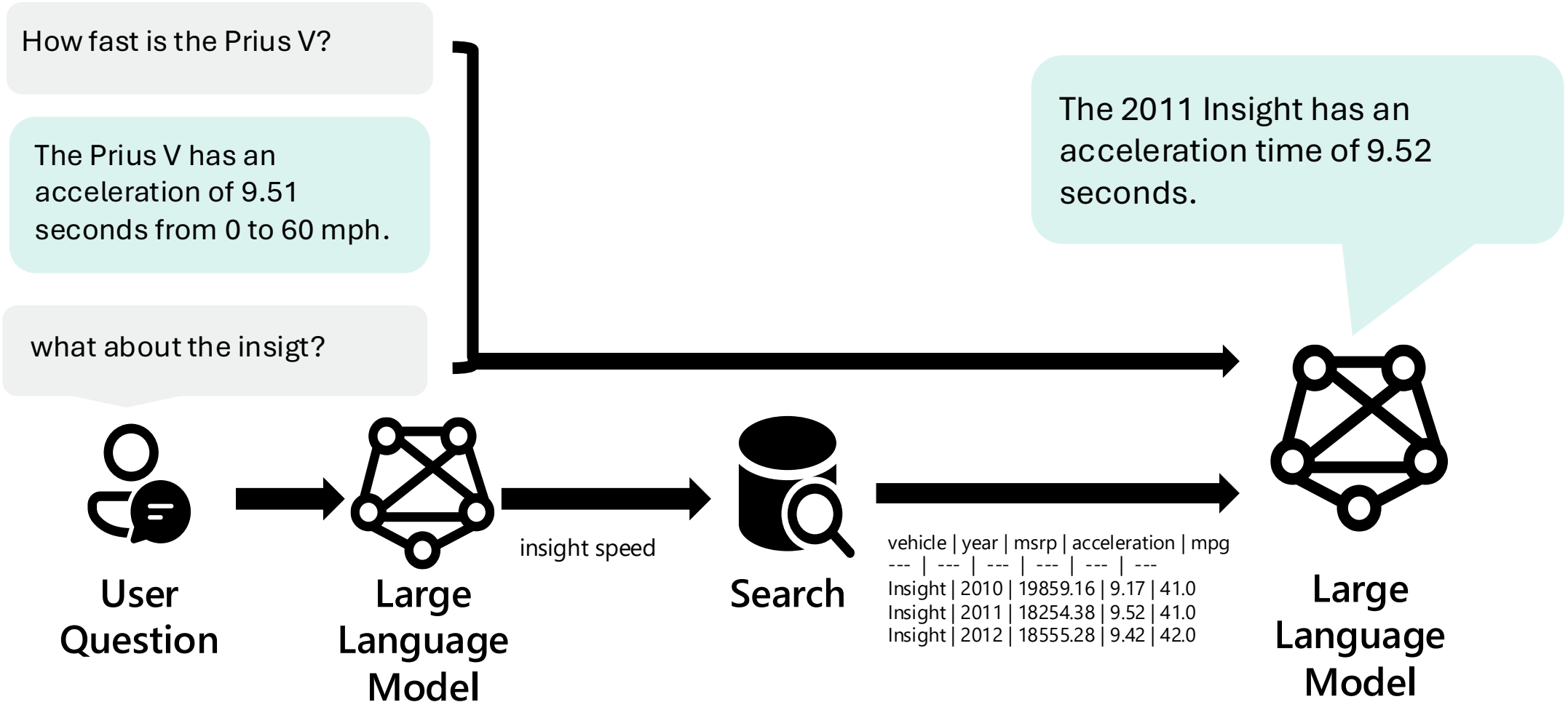
Simple RAG flow



RAG flow with multiturn support



RAG flow with multiturn + query rewriting



RAG data source types

Database rows (Structured data)

You need a way to **vectorize** target columns with an **embedding model**.

You need a way to **search** the vectorized rows.

Documents (Unstructured data)

PDFs, docx, pptx, md, html, images

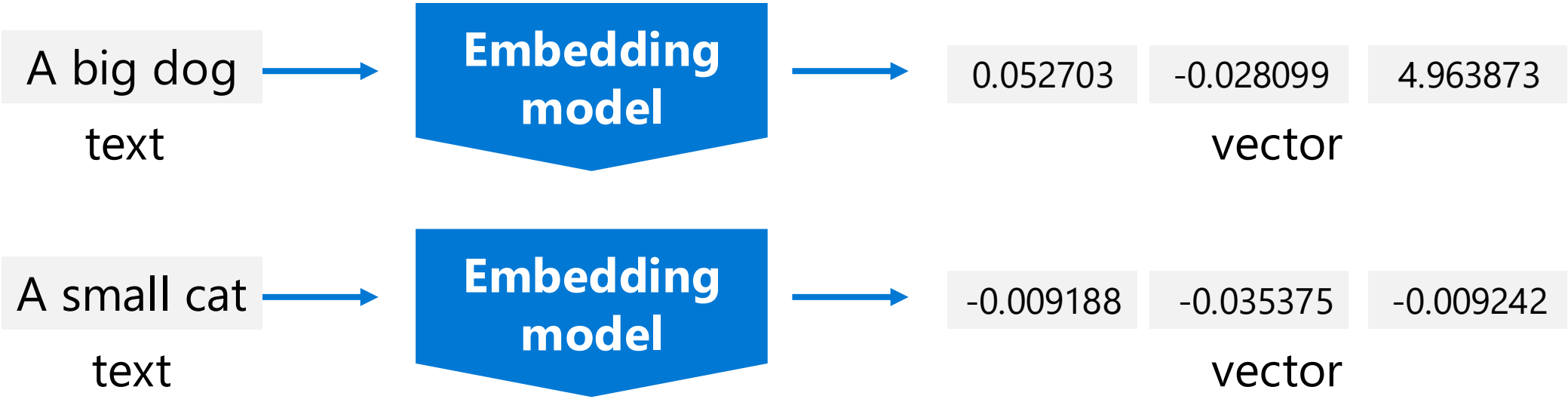
You need an ingestion process for **extracting**, **splitting**, **vectorizing**, and **storing** document chunks.

You need a way to **search** the vectorized **chunks**.

Vector embeddings

Vector embeddings

A **vector embedding** represents an input (text/phrase/image/etc) as a list of numbers, a vector in a multi-dimensional space.



A vector embedding model is trained specifically to convert inputs into a vector, using massive training data sets and architectures similar to LLM models.

Vector embedding models

Different embedding models output different embeddings with varying lengths. The goal of all models is to capture the meaning.

Embedding model	Encodes	Vector length
Sbert (Sentence-Transformers)	text (up to ~400 words)	768
OpenAI text-embedding-ada-002	text (up to 8191 tokens)	1536
OpenAI text-embedding-3-small	text (up to 8191 tokens)	256 - 1536
OpenAI text-embedding-3-large	text (up to 8191 tokens)	256 - 3072
nomic-embed-text	text (up to 8191 tokens)	768
Azure AI Vision	image or text	1024

MTEB: https://huggingface.co/spaces/mteb/leaderboard

Generating an embedding in JavaScript

Configure OpenAI SDK (e.g. with models hosted on Microsoft Foundry):

```
import OpenAI from "openai";

const client = new OpenAI({
  apiKey: process.env.AZURE_OPENAI_API_KEY,
  baseURL: process.env.AZURE_OPENAI_ENDPOINT});
```

Generate embedding:

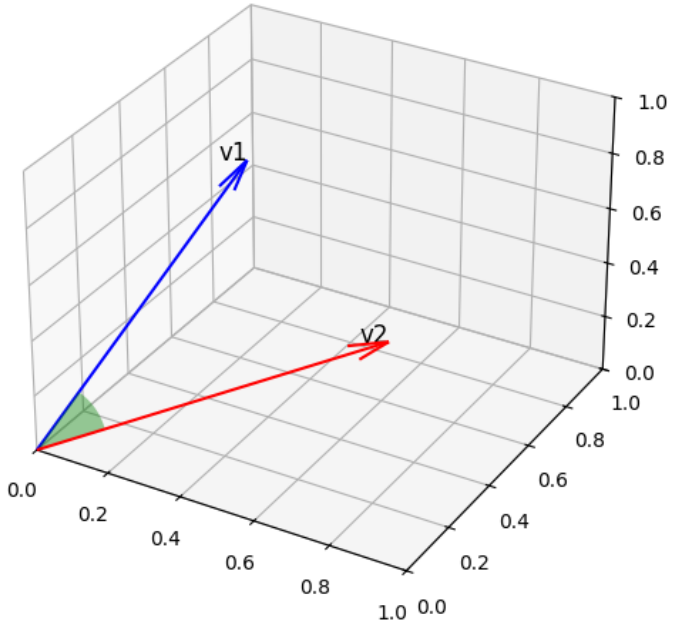
```
const response = await client.embeddings.create({
  model: "text-embedding-3-small",
  dimensions: 1536,
  input: "hello world"
});
```

Measuring vector similarity

We compute embeddings so that we can calculate similarity between inputs.

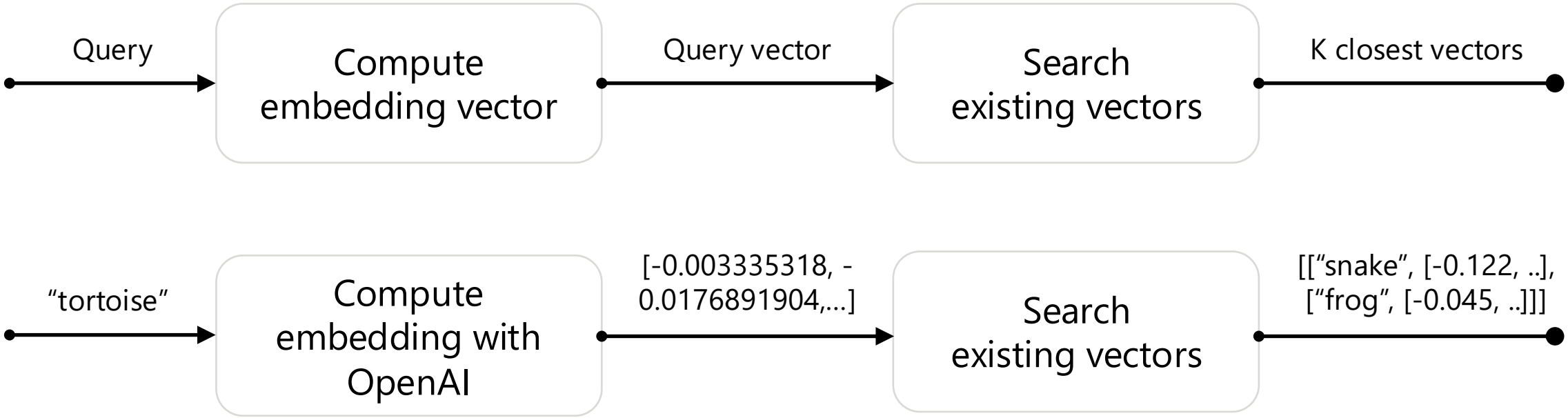
The most common distance measurement is **cosine similarity**.

```
function cosineSimilarity(v1, v2) {

  const dotProduct = v1.reduce(
    (sum, a, i) => sum + a * v2[i], 0);

  const magnitude =
    Math.sqrt(v1.reduce((sum, a) => sum + a ** 2, 0)) *
    Math.sqrt(v2.reduce((sum, a) => sum + a ** 2, 0));

  return dotProduct / magnitude;
}
```



Vector search

1. Compute the embedding vector for the query
2. Find K closest vectors for the query vector
   - Search exhaustively or using approximations



ANN (Approximate Nearest Neighbor) search

There are multiple ANN search algorithms that can speed up search time.

Algorithm	JS library	Database support
HNSW	hnswlib-node	PostgreSQL pgvector extension Azure AI Search Chromadb Weaviate
DiskANN	(server-side only)	Cosmos DB, Azure SQL, Azure Database for PostgreSQL
IVFFlat	faiss-node	PostgreSQL pgvector extension
Faiss	faiss-node	None, in-memory index only

Optimal search strategy

Vector search is NOT enough!

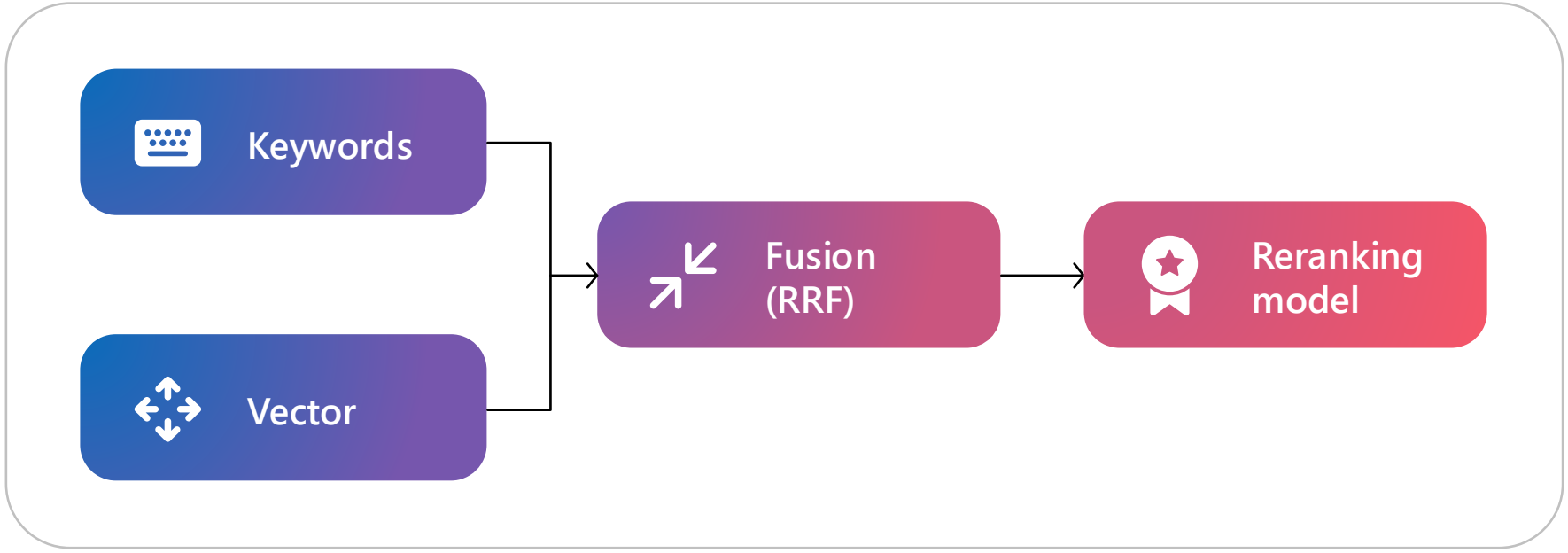
Query type	Keyword [NDCG@3]	Vector [NDCG@3]	Hybrid [NDCG@3]	Hybrid + Semantic ranker [NDCG@3]
Concept seeking queries	39	45.8	46.3	59.6
Fact seeking queries	37.8	49	49.1	63.4
Exact snippet search	51.1	41.5	51	60.8
Web search-like queries	41.8	46.3	50	58.9
Keyword queries	79.2	11.7	61	66.9
Low query/doc term overlap	23	36.1	35.9	49.1
Queries with misspellings	28.8	39.1	40.6	54.6
Long queries	42.7	41.6	48.1	59.4
Medium queries	38.1	44.7	46.7	59.9
Short queries	53.1	38.8	53	63.9

Source: Outperforming vector search with hybrid + reranking

Hybrid search: The best of both worlds!

Complete search stacks do better:

- Hybrid retrieval (keywords + vectors) > pure-vector or keyword
- Hybrid + Reranking > Hybrid



Let's break those down...

Keyword search

Best for: exact phrase matches

Documents are stored in an **inverted index** which can be queried efficiently.

ID	Name	Description
335	Soaker Hose 25-foot	Porous soaker hose for efficient water delivery directly to plant roots.
334	Garden Hose 50-foot	Heavy-duty garden hose with brass fittings and kink-resistant construction.

Term	Document List
hose	335, 336
foot	335, 336
garden	335
25	336

"25 foot hose" → **Full text search of name + description** →

ID	Document List	Relevance
335	Soaker Hose 25-foot	0.066
336	Expandable Hose 100-foot	0.046
334	Garden Hose 50-foot	0.046
312	Drain Snake 25-foot	0.040

Tip: **BM25** is the best algorithm, but other inverted index algorithms are also effective.

Vector search

Best for: conceptually similar matches

1. **Encode queries as embeddings** (lists of floating point numbers)
2. **Find semantically similar documents by comparing embeddings**

Query

"garden watering supplies"

Compute embedding vector

Query vector

[-0.003335318, -0.0176891904,...]

Search existing vectors

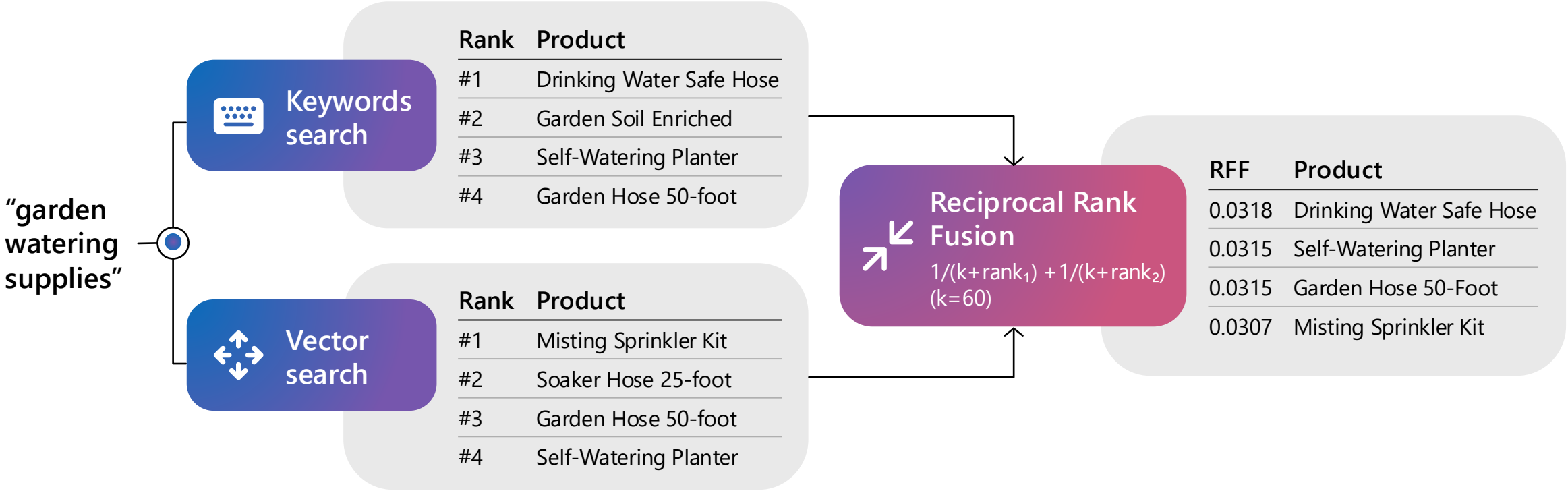
K closest vectors

ID	Name	Cosine Similarity
335	Soaker Hose 25-foot	0.51
336	Garden Hose 50-foot	0.51
338	Retractable Hose Hanger	0.46

Tip: Use approximation algorithm (**HNSW** or **DiskANN**) for more efficient search.

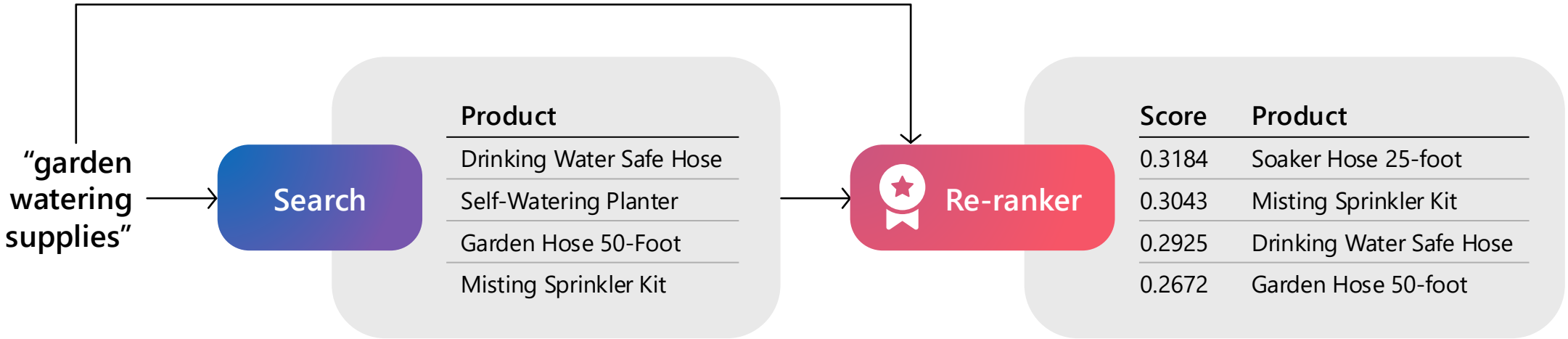
RRF: Reciprocal Rank Fusion

Using text **and** vector search? Merge results using RRF to rank results based off their relative ranks.



Re-ranking

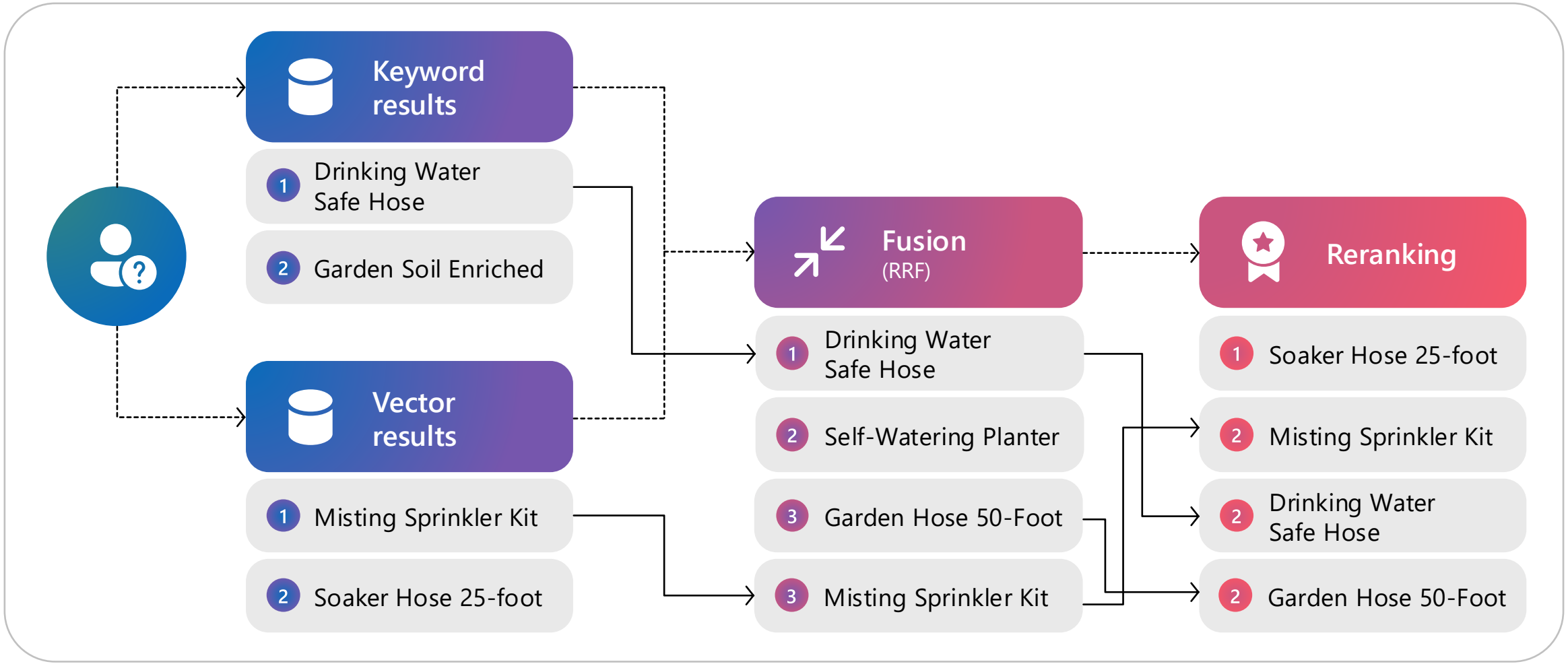
A re-ranker model gives documents a relevance score according to the query.



Tip: Cross-encoder models (like Cohere Reranker) are best, but LLMs can be used as well.

A complete hybrid search flow

Question: "garden watering supplies"



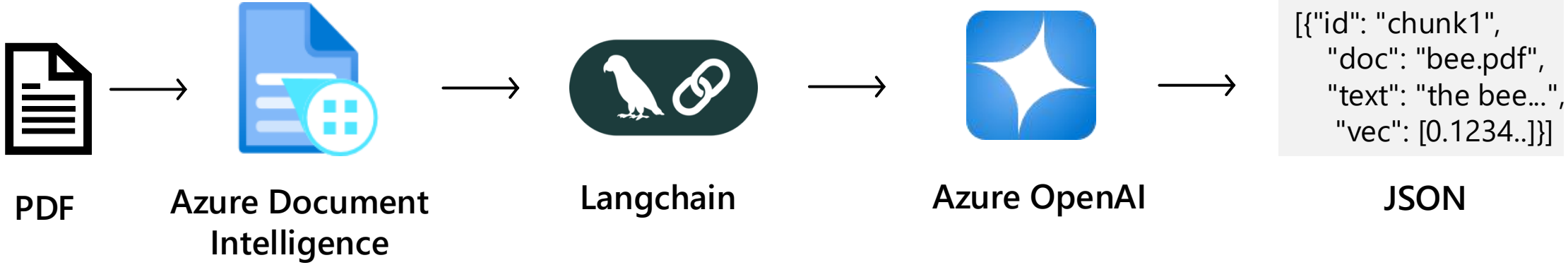
Hybrid search is supported on many Azure databases

Database	Vector search?	Keyword search?	RRF?	Re-ranker?
Azure Database for PostgreSQL	✓ pgvector HNSW/IVF/DiskANN	✓ fts	✓	✓ azure_ai.rank()
Azure AI Search	✓ HNSW	✓ BM25	✓	✓
Azure Cosmos DB	✓ VectorDistance() DiskANN	✓ BM25	✓	✓ (Public preview)
Azure SQL	✓ vector_distance() DiskANN	✓ BM25	✓	✗

Data ingestion

RAG document ingestion

For long/unstructured documents, we need an ingestion flow such as this one:



Extract text from PDF

Other options for this step: Azure Document Intelligence, Langchain document loaders, OCR services, Unstructured, etc.

Split data into chunks

Split text based on sentence boundaries and token lengths.

You could also use "semantic" splitters and your own custom splitters.

Vectorize chunks

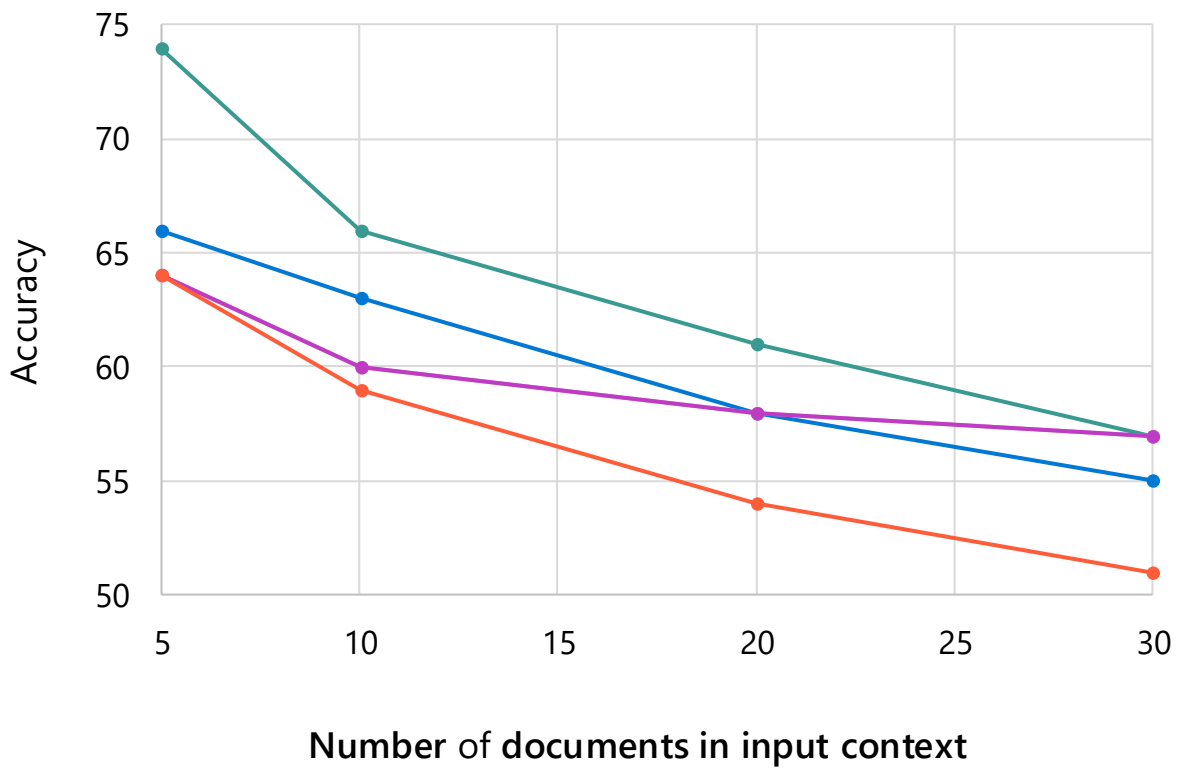
Compute embeddings using embedding model of your choosing.

Store chunks

This is where you'd typically use a search service like Azure AI Search or a database like PostgreSQL or Cosmos DB.

Why do we need to split documents?

1. LLMs have limited context windows (128K – 1M)
2. When an LLM receives too much information, it can get easily distracted by irrelevant details.
3. The more tokens you send, the higher the cost, the slower the response.



Source: Lost in the Middle: How Language Models Use Long Contexts, Liu et al. arXiv:2307.03172

Optimal size of document chunk

How big should chunks be?

# of tokens per chunk	Recall@50
512	**42.4**
1024	37.5
4096	36.4
8191	34.9

Source: aka.ms/ragrelevance

A **token** is the unit of measurement for an LLM's input/output. ~1 token/word for English, higher ratios for other languages.
More on token ratios: aka.ms/genai-cjk

Where to split chunks?

Chunk boundary strategy	Recall@50
Break at token boundary	40.9
Preserve sentence boundaries	42.4
10% overlapping chunks	43.1
25% overlapping chunks	**43.9**

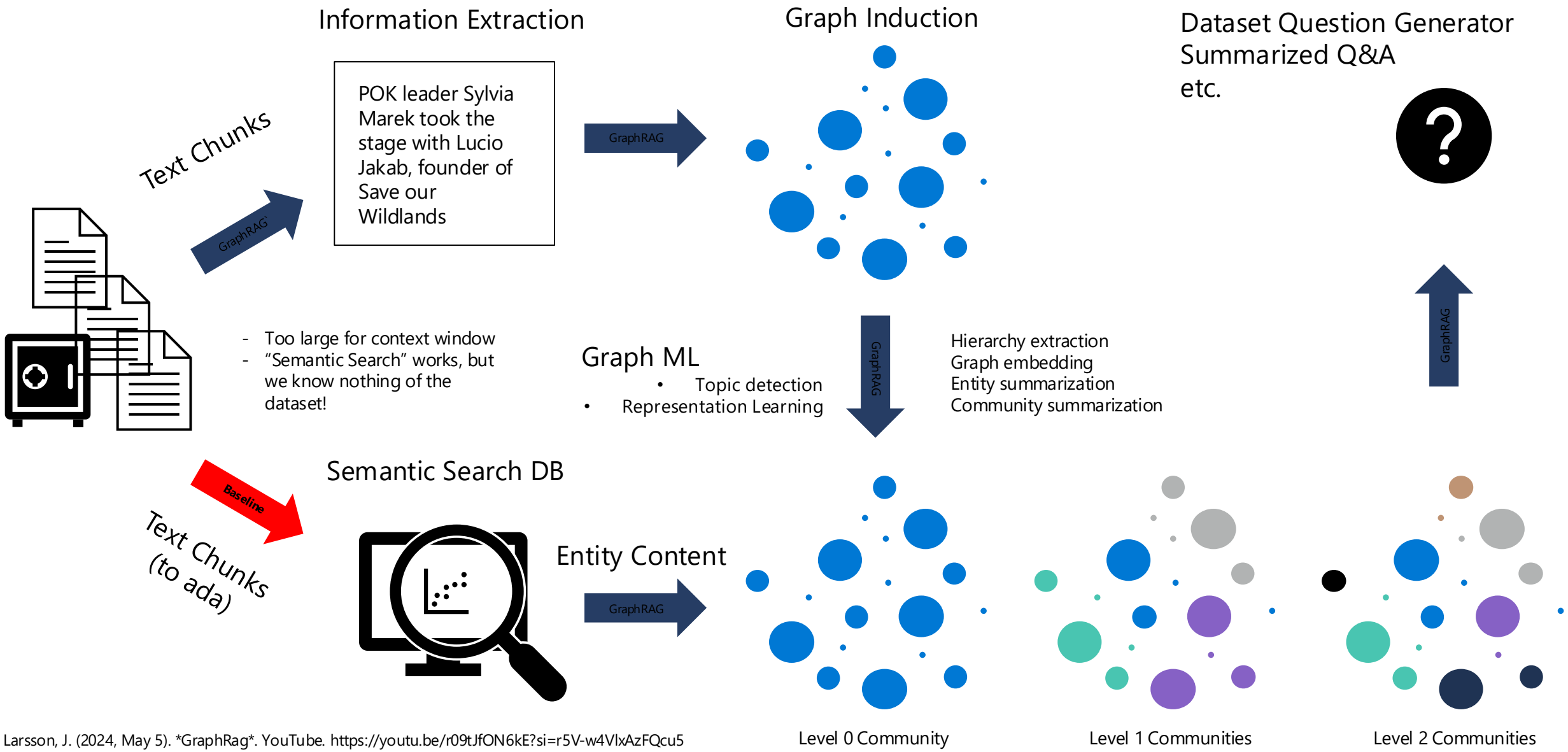
Source: aka.ms/ragrelevance

A chunking algorithm should also consider tables, and avoid splitting tables when possible.

More RAG approaches

Graph RAG

https://www.microsoft.com/research/project/graphrag/

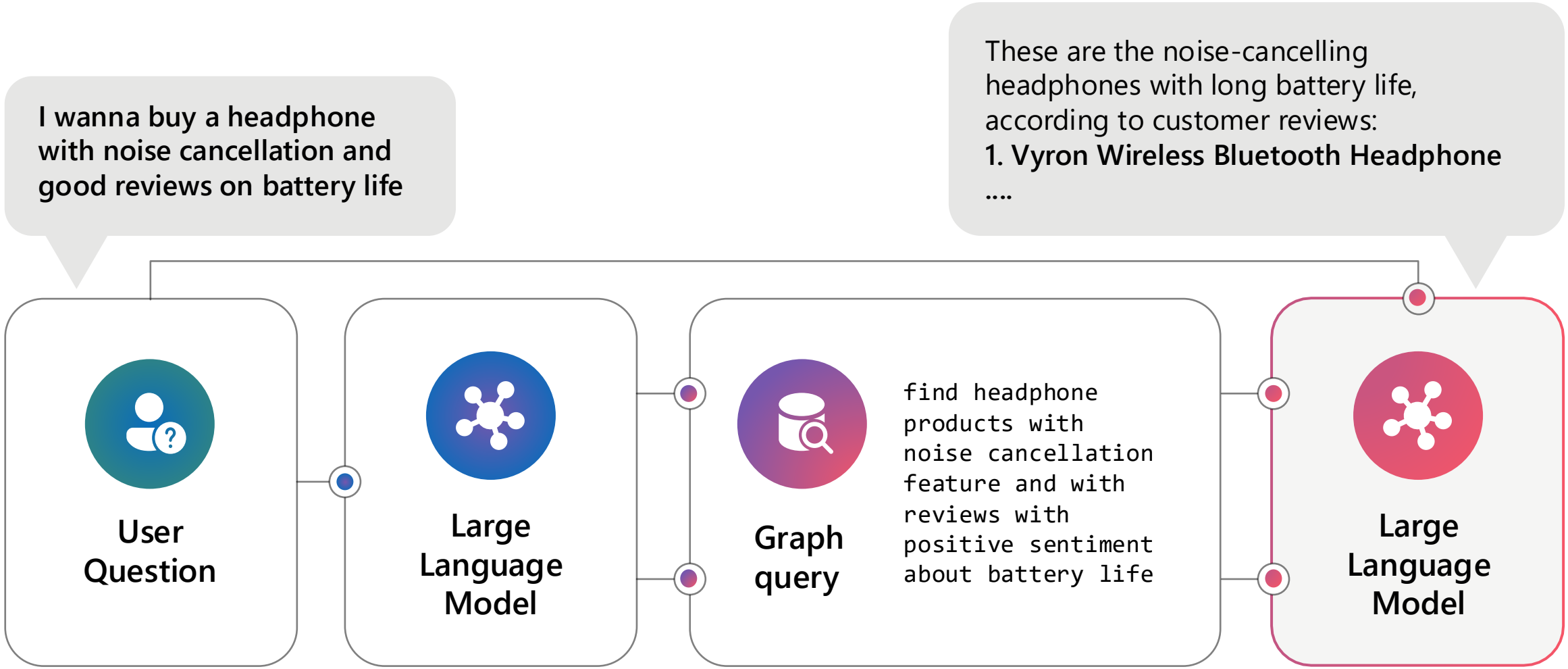


Larsson, J. (2024, May 5). *GraphRag*. YouTube. https://youtu.be/r09tJfON6kE?si=r5V-w4VlxAzFQcu5

RAG with graph queries

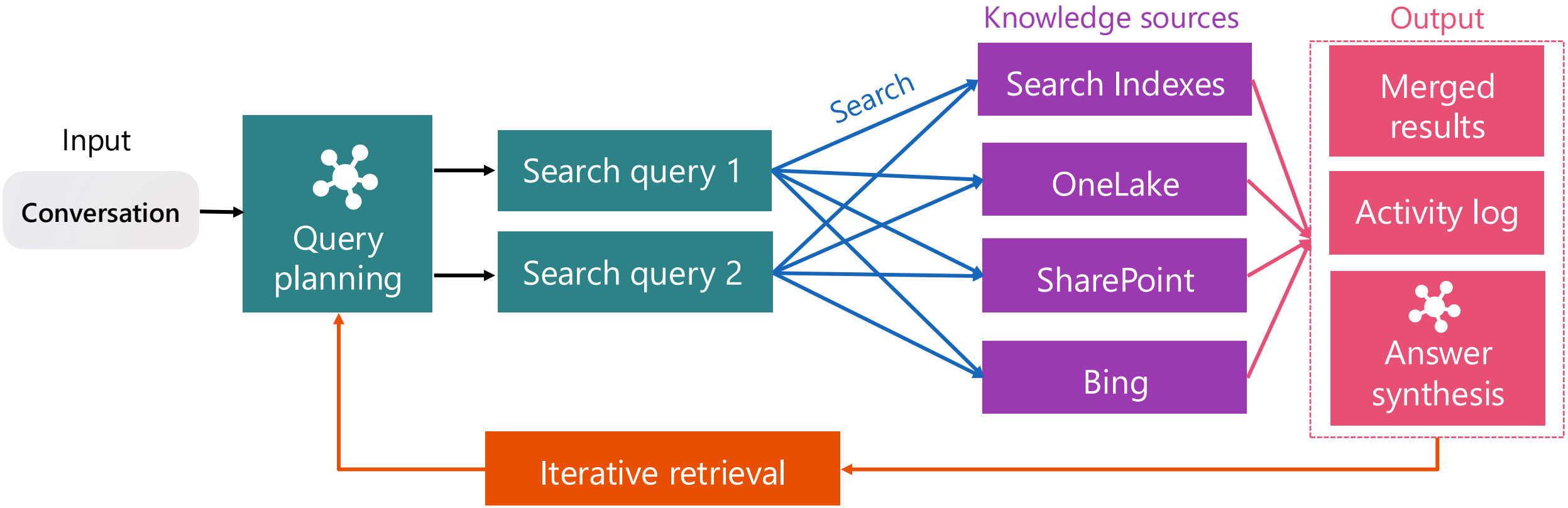
https://aka.ms/agentic-shop

Azure PostgreSQL + AGE can use Cypher graph queries to query across relationships:



Agentic RAG across multiple sources

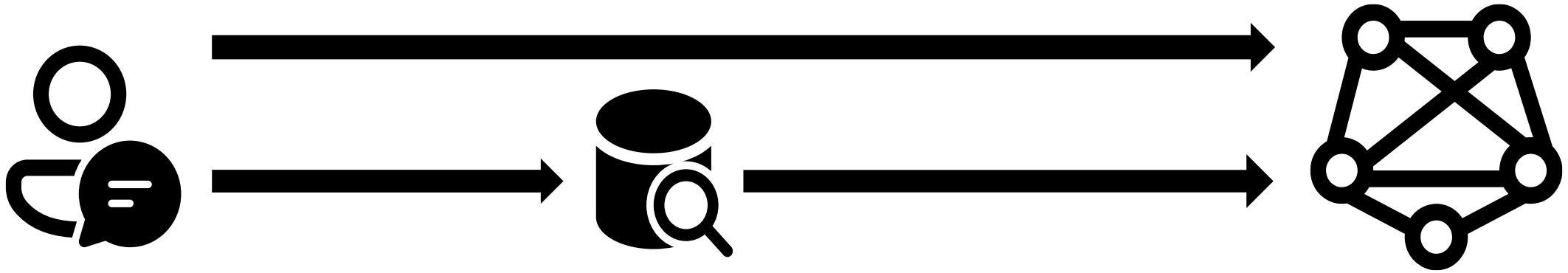
Azure AI Search knowledge bases use LLMs for query planning and source selection:



Search step uses hybrid search with semantic re-ranking when supported by the knowledge source

Evaluation

What affects the quality of a RAG flow?



Question

Search

- Search engine (ie. Azure AI Search)
- Search query cleaning
- Search options (hybrid, vector, reranker)
- Additional search options
- Data chunk size and overlap
- Number of results returned

Large Language Model

- System prompt
- Language
- Message history
- Model (ie. GPT 3.5)
- Temperature (0-1)
- Max tokens

Are the answers high quality?

- Are they correct? (relative to the knowledge base)
- Are they clear and understandable?
- Are they formatted in the desired manner?

Do the perks cover underwater activities?

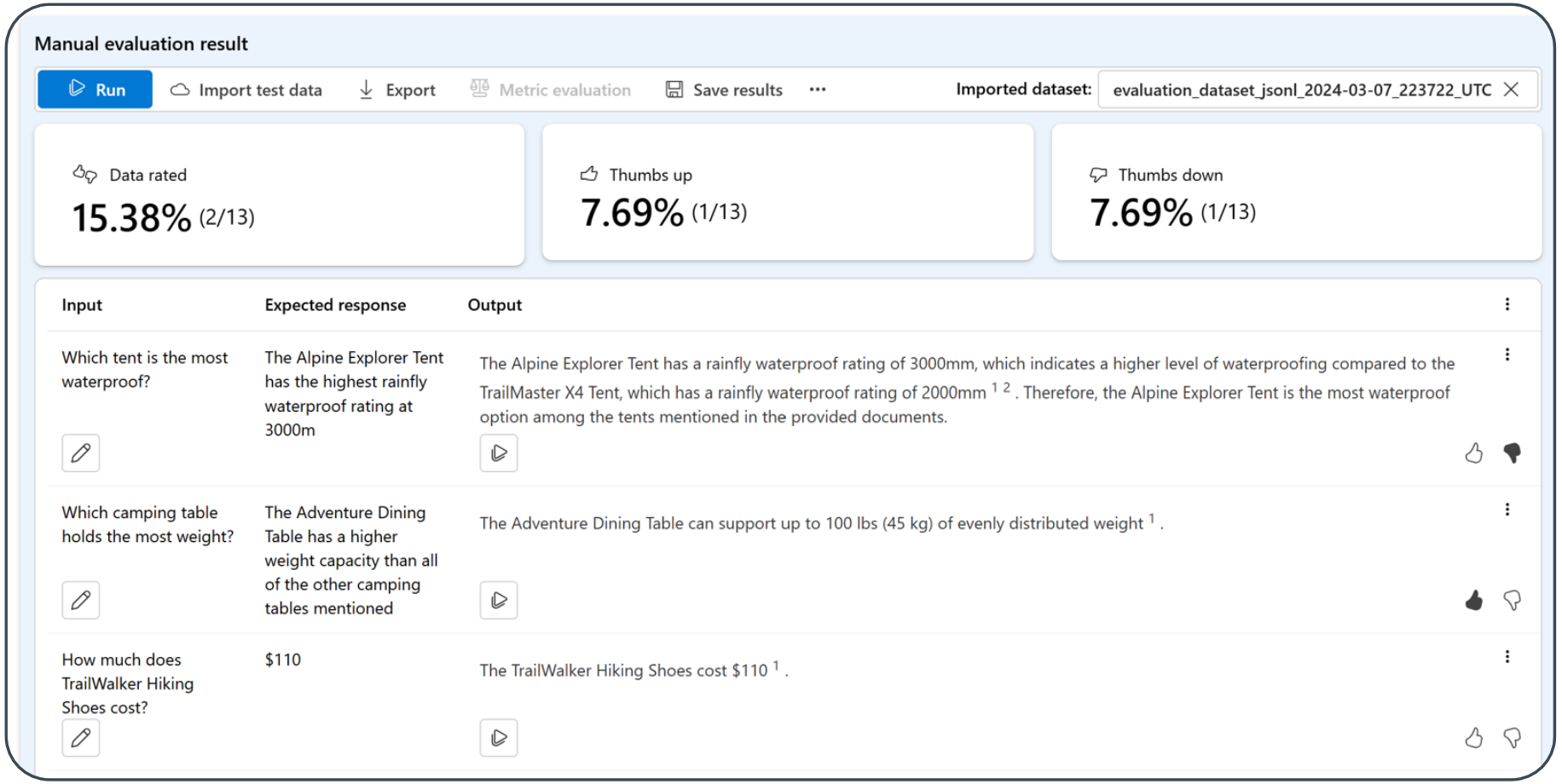
Yes, according to the information provided in the PerksPlus.pdf document, underwater activities such as scuba diving are covered under the program.

Yes, the perks provided by the PerksPlus Health and Wellness Reimbursement Program cover a wide range of fitness activities, including underwater activities such as scuba diving. The program aims to support employees' physical health and overall well-being, so it includes various lessons and experiences that promote health and wellness. Scuba diving lessons are specifically mentioned as one of the activities covered under PerksPlus. Therefore, if an employee wishes to pursue scuba diving as a fitness-related activity, they can expense it through the PerksPlus program.

Yes, underwater activities are included as part of the PerksPlus program. Some of the underwater activities covered under PerksPlus include scuba diving lessons [PerksPlus.pdf#page=3].

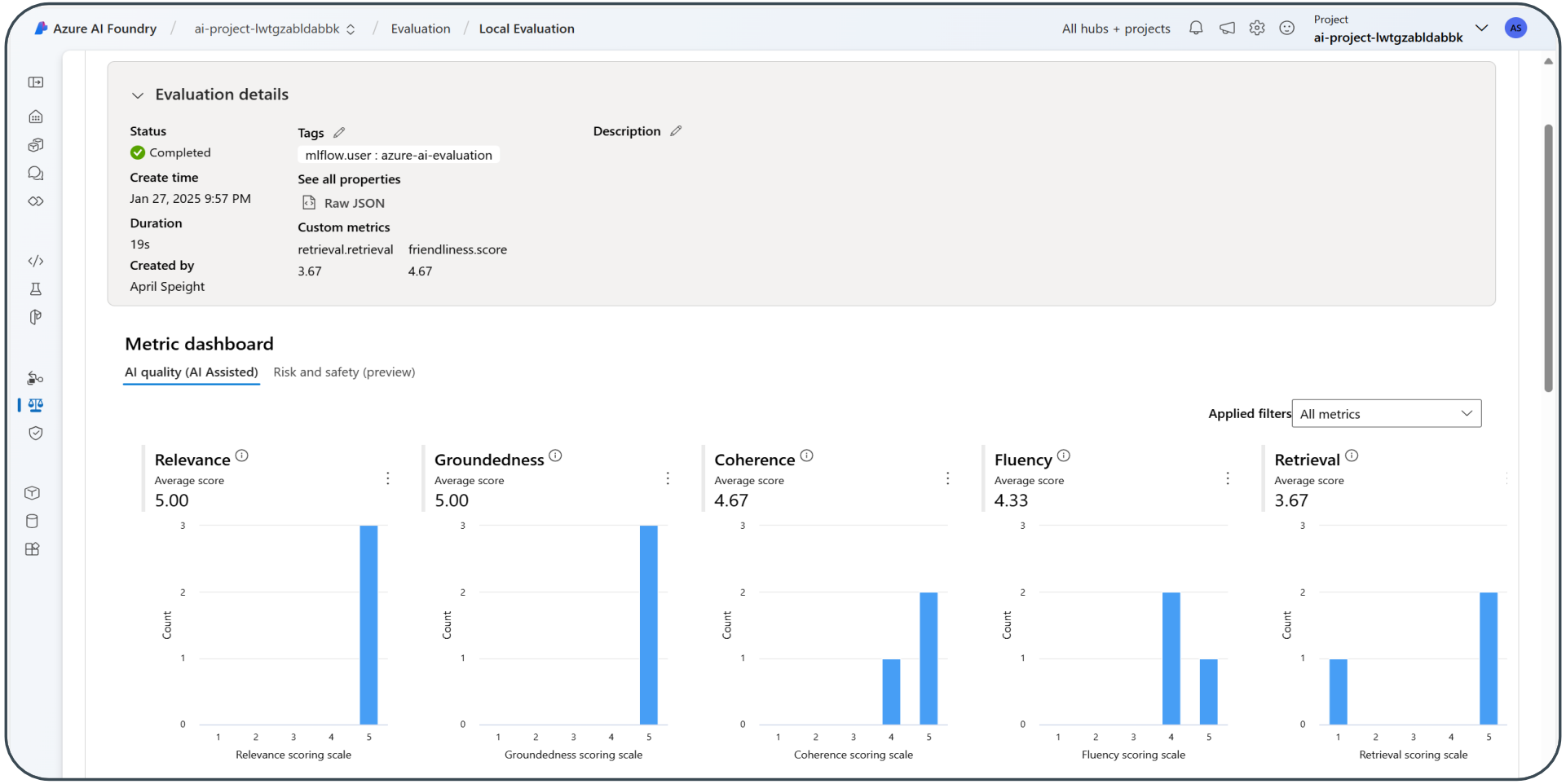
Manual evaluation

Humans can spot-check output for small data sets and annotate issues:



Automated evaluation

AI models can measure output performance at scale across larger data sets:

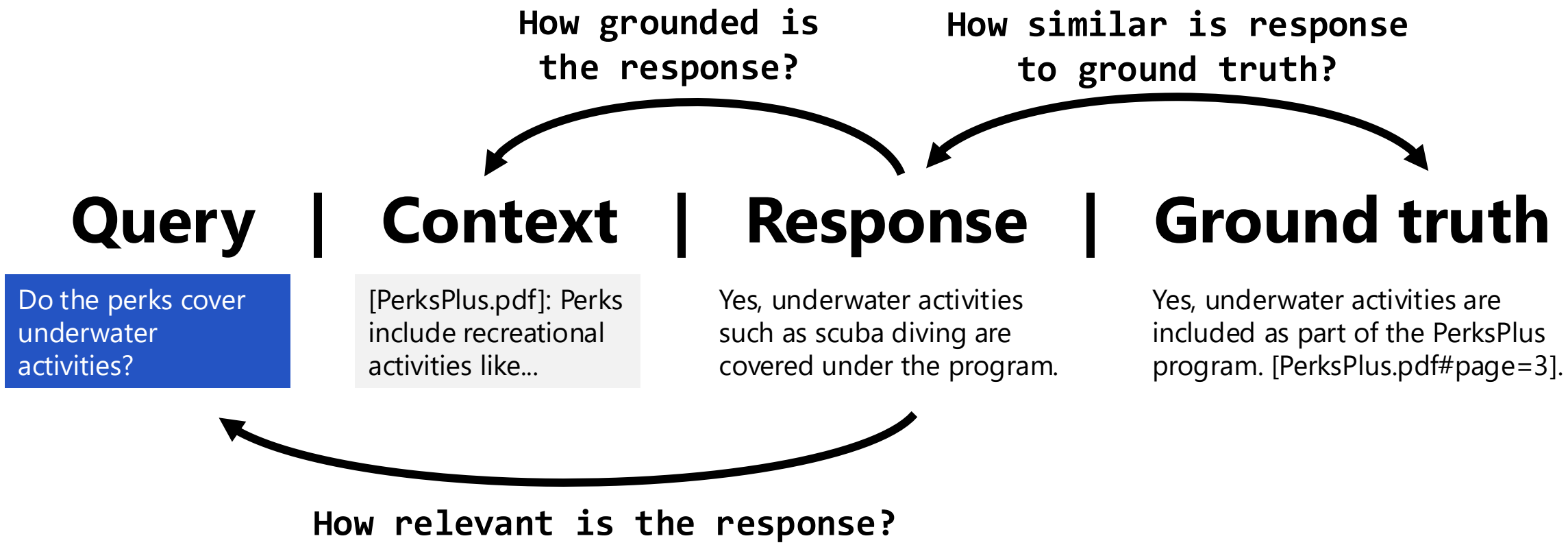


Automated evaluation frameworks

Framework	Author	Language	Cloud hosted?
azure-ai-evaluation / Microsoft.Extensions.AI.Evaluation	Microsoft	Python .NET	Optional
RAGAS	ExplodingGradients	Python	None
DeepEval	ConfidentAI	Python	Optional
Langsmith	Langchain	Python	Required
Promptfoo	Promptfoo	JavaScript	Optional

LLM-based quality evaluation

Use an LLM-as-a-judge to score the output from the RAG flow:



Run evaluation across a large data set that represents a range of user queries.

Review evaluation metrics

Compare metrics across runs so you can see which increased/decreased:

folder	groundedness	%	relevance	%	coherence	%	citation %	length
badprompt_ignoresources_1	4.27	0.81	4.10	0.77	4.60	0.89	0.00	1255.44
baseline_1	4.79	0.93	4.78	0.94	4.84	0.96	0.98	648.97
baseline_2	4.89	0.96	4.76	0.94	4.80	0.96	0.98	659.05
baseline_3	4.83	0.94	4.74	0.93	4.80	0.95	0.98	659.39
generalprompt_1	4.86	0.95	4.84	0.96	4.93	0.98	0.00	1164.14
refinedprompt_1	4.93	0.97	4.88	0.96	4.96	0.99	0.99	1430.57
refinedprompt_2	4.87	0.95	4.86	0.96	4.95	0.99	0.99	1398.83
refinedprompt_trimmed_1	4.90	0.97	4.83	0.95	4.93	0.98	0.99	969.89
refinedprompt_trimmed_2	4.91	0.97	4.89	0.97	4.89	0.98	0.99	975.07
refinedprompt_trimmed_3	4.95	0.98	4.84	0.96	4.92	0.98	0.99	965.01
refinedprompt_trimmed_4	4.93	0.97	4.89	0.97	4.86	0.96	0.98	962.78

Tip: Include metrics like latency and length, since a quality increase might not be worth an increase in cost/latency.

Compare individual answers

Once you see the difference in metrics, compare answers with changed scores:

```
What is the importance of choosing the right provider in getting the most value out of your health
insurance plan?

baseline_1

    Choosing the right provider is important
    for getting the most value out of your
    health insurance plan. In-network
    providers charge lower rates than
    out-of-network providers, so choosing an
    in-network provider can help you save
    money on your healthcare costs
    [Northwind_Standard_Benefits_Details.pdf#
    page=5][Northwind_Health_Plus_Benefits_De
    tails.pdf#page=4]. In addition,
    continuity of care is an important

groundedness  relevance  coherence
5             5          5

baseline_2

    Choosing the right provider is important
    in getting the most value out of your
    health insurance plan for several
    reasons:

     1. Cost savings: In-network providers
        charge lower rates than
        out-of-network providers, so choosing
        an in-network provider can help you
        save money on your healthcare costs.
        [Northwind_Standard_Benefits_Details.

groundedness  relevance  coherence
5             5          5
```

Lessons learnt from evaluating RAG apps

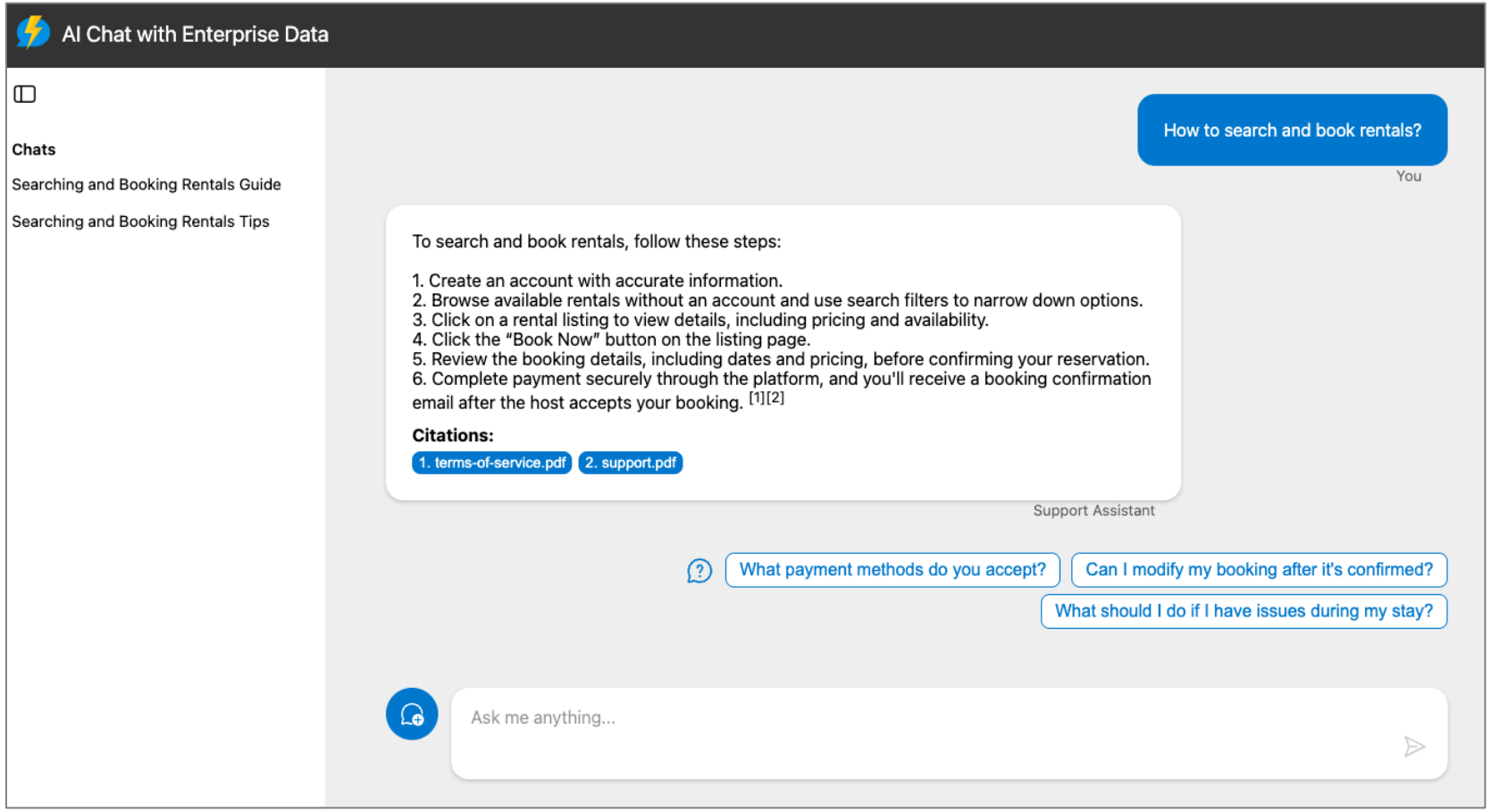
- RAG on data that you know; LLMs can be convincingly wrong.
- What works better for 3 Qs doesn't always work better for 200.
- Don't trust absolute metrics, trust relative metrics.
- Vector search can be noisy! Use wisely.
- Your model choice makes a huge difference.
- Remove the fluff from prompts; it doesn't matter.

Subscribe to Hamel Husain for evaluation tips:

https://hamel.dev/blog/posts/evals-faq/

Code walkthrough

Open source RAG solution



Data type:
Documents (PDF)

Search:
Cosmos DB

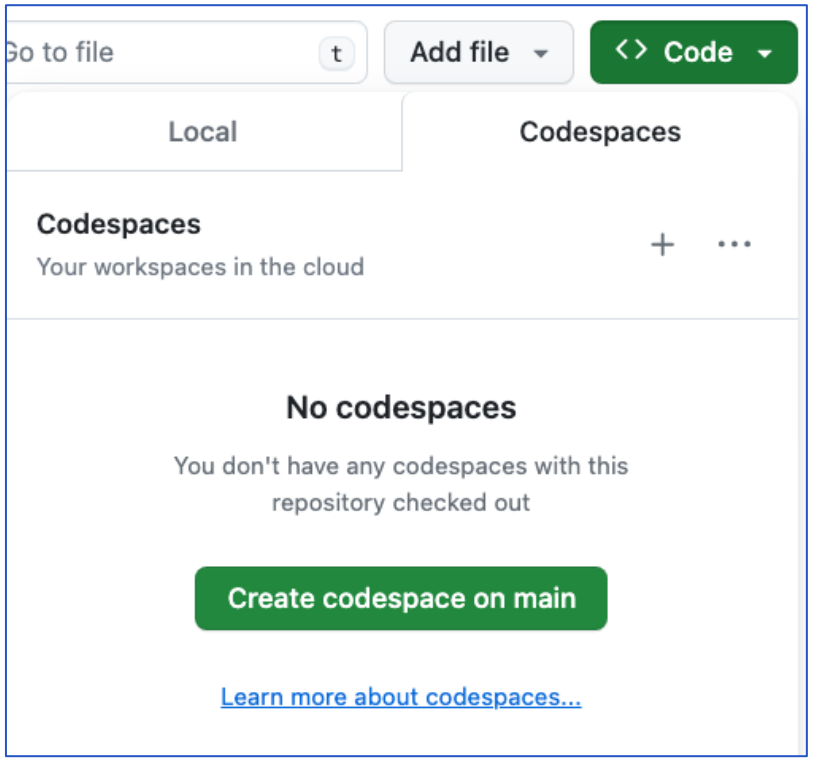
LLM:
Azure OpenAI (GPT 4o)
Ollama (Llama3.1)

Features:
Multi-turn streaming chat
Chat session history

https://github.com/azure-samples/serverless-chat-langchainjs/

Opening the project: 3 options

- GitHub Codespaces →
- VS Code with Dev Containers extension
- Your Local Environment
  - Node LTS
  - Azure Developer CLI
  - Azure Functions Core Tools
  - Git



https://github.com/azure-samples/serverless-chat-langchainjs/?tab=readme-ov-file#getting-started

Deploying with the Azure Developer CLI (azd)

Prerequisites:

- Azure account and subscription
- Azure account permissions:
  - Microsoft.Authorization/roleAssignments/write
  - Microsoft.Resources/deployments/write on subscription level

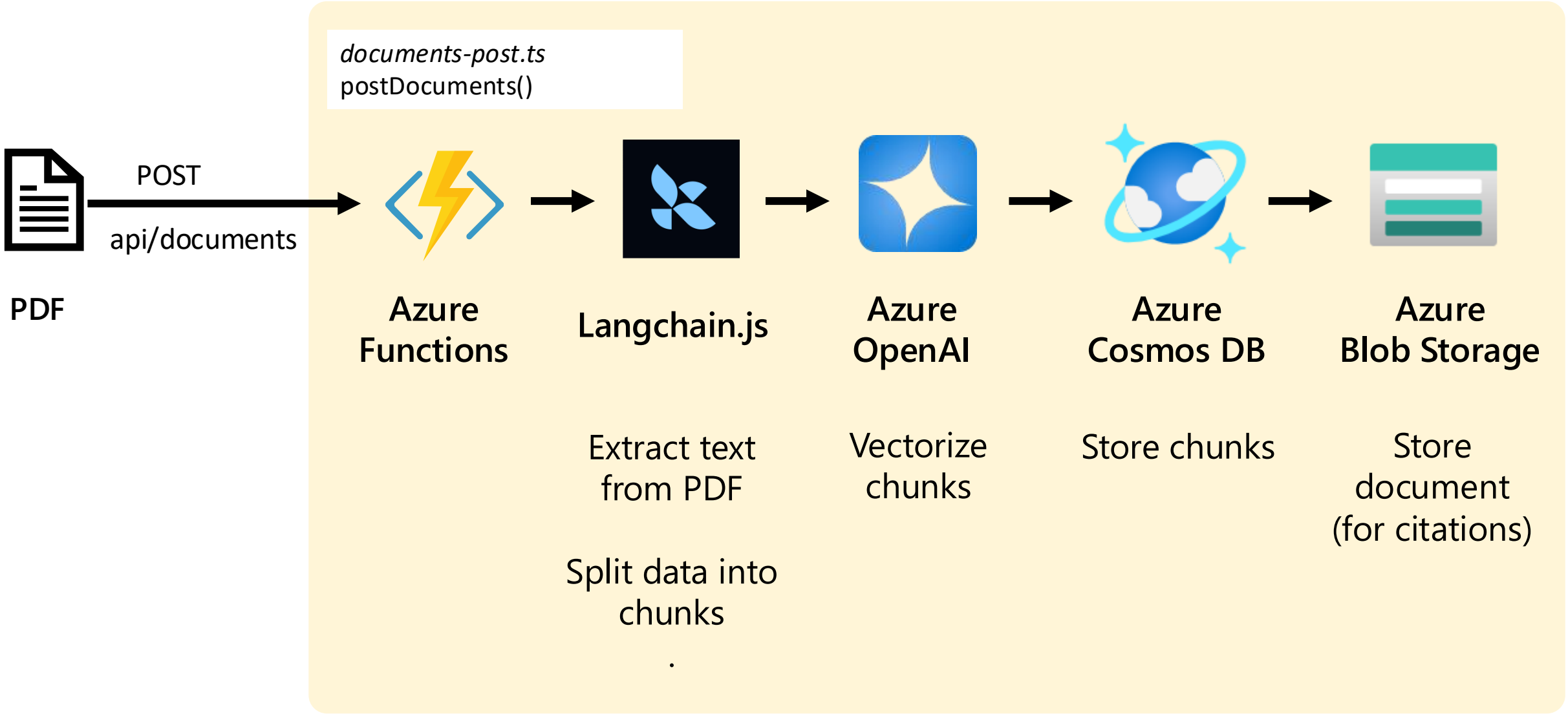
Login to your Azure account:

```
azd auth login
```

Provision resources, deploy app, and ingestion documents:

```
azd up
```

Data ingestion process



Application architecture

Typescript frontend

(Lit + Vite)

chat.ts
ChatComponent

→

api.ts
getCompletion()

POST

chats/stream

→

Azure Static Web Apps

NodeJS backend

(Azure Functions v4 + Langchain.js)

chats-post.ts
postChats()

// Search the DB
retriever.invoke()

Cosmos DB

// Combine prompt + results
createStuffDocumentsChain()

Azure OpenAI

Azure Functions

Next steps

Deploy the app and customize for your own data!
https://github.com/Azure-Samples/serverless-chat-langchainjs

Check out my RAG deep dive series:
https://aka.ms/ragdeepdive/watch

Post in the Foundry Discord when you have questions:
https://aka.ms/foundry/discord